TALLER No.6 REPASO ESTRUCTURAS DE DATOS BIDIMENSIONALES - MATRICES

NOMBRE DEL COMPONENTE FORMATIVO	Pseint y DEV C++
BREVE DESCRIPCIÓN	Propósito de la actividad: Desarrollar la capacidad de analizar problemas, almacenar conjuntos de datos en estructuras bidimensionales (matrices) y manipularlos mediante recorridos anidados, operaciones por filas/columnas y algoritmos de búsqueda en PSeInt.
PALABRAS CLAVE	Programación, Código, análisis.

A. TABLA DE CONTENIDOS:

1. DATOS GENERALES

Tema: Matrices y Arreglos Bidimensionales
Lenguaje: PSeInt (Pseudocódigo)
Duración sugerida: 4 horas
Modalidad: Práctica individual / guiada
Nivel: Intermedio
Evidencia a entregar: Archivos de pseudocódigo (.psc) y documento de evidencias con ejecución y reflexión.

2. OBJETIVOS DE APRENDIZAJE

• Comprender el concepto de matriz / arreglo bidimensional en PSeInt como una tabla de filas y columnas.
• Declarar (Dimension), inicializar y acceder a los elementos de una matriz mediante dos índices [fila, columna].
• Recorrer matrices utilizando ciclos Para anidados.
• Realizar operaciones matemáticas sobre filas específicas, columnas o la matriz completa.
• Aplicar la lógica de matrices en la resolución de problemas reales (ej. tableros, calificaciones, inventarios).

3. CONCEPTOS CLAVE: ¿QUÉ ES UNA MATRIZ EN PSEINT?

Una matriz (o arreglo bidimensional) es una estructura de datos que permite almacenar información en un formato de tabla o cuadrícula, organizada en filas y columnas. En PSeInt, se definen con la palabra clave Dimension indicando dos tamaños: [cantidad_filas, cantidad_columnas]. Para acceder a un dato, siempre se necesitan dos coordenadas.

```
// Declaración de una matriz de 3 filas y 4 columnas
Definir tabla Como Entero;
Dimension tabla[3, 4];

// Asignación de valores (Fila 1, Columna 2)
tabla[1, 2] <- 15;

// Acceso a un elemento
Escribir tabla[1, 2]; // Muestra 15
```

Recuerda: El primer índice siempre representa la FILA y el segundo la COLUMNA. En esta guía asumiremos que los índices inician en 1.

PARTE A. CREACIÓN, LECTURA Y RECORRIDO DE MATRICES

Ejercicio 1. Matriz estática y acceso simple

Cree una matriz de 2x2. Asigne manualmente un valor diferente a cada una de las 4 posiciones. Luego, imprima los valores ubicados en la diagonal principal (posición [1,1] y [2,2]).

```
1  Algoritmo Ejercicio1_Matriz
2      Definir mi_matriz Como Entero;
3      Dimension mi_matriz[2, 2];
4
5      // Asignación manual
6      mi_matriz[1, 1] ← 10;
7      mi_matriz[1, 2] ← 20;
8      mi_matriz[2, 1] ← 30;
9      mi_matriz[2, 2] ← 40;
10
11     // Imprimir diagonal principal
12     Escribir "Elemento en [1,1]: ", mi_matriz[1, 1];
13     Escribir "Elemento en [2,2]: ", mi_matriz[2, 2];
14 FinAlgoritmo
```

Cuando sale fuera de rango:

numero1.psc

```
Algoritmo DiagonalMatriz
    Definir matriz Como Entero;
    Dimension matriz[2,2];

    // Asignación manual de valores diferentes
    matriz[0,0] ← 10;
    matriz[0,1] ← 20;
    matriz[1,0] ← 30;
    matriz[1,1] ← 40;

    // Impresión de los elementos de la diagonal principal
    Escribir "Valores de la diagonal principal:";
    Escribir "Posicion [0,0]: ", matriz[0,0];
    Escribir "Posicion [1,1]: ", matriz[1,1];
FinAlgoritmo
```

numero1.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    // Declaración de la matriz de 2 filas y 2 columnas
    int mi_matriz[2][2];

    // Asignación manual de valores
    mi_matriz[0][0] = 10; // Equivalente a la Fila 1, Columna 1
    mi_matriz[0][1] = 20; // Equivalente a la Fila 1, Columna 2
    mi_matriz[1][0] = 30; // Equivalente a la Fila 2, Columna 1
    mi_matriz[1][1] = 40; // Equivalente a la Fila 2, Columna 2

    // Imprimir los elementos de la diagonal principal
    cout << "Elemento en la posicion [0][0] (Diagonal principal): " << mi_matriz[0][0] << endl;
    cout << "Elemento en la posicion [1][1] (Diagonal principal): " << mi_matriz[1][1] << endl;

    return 0;
}
```

Ejercicio 2. Llenado dinámico e impresión en formato tabla

Solicite al usuario que ingrese los datos para llenar una matriz de 3 filas y 3 columnas. Luego, imprima la matriz en pantalla respetando el formato de cuadrícula (filas y columnas).

```
Algoritmo Ejercicio2_Matriz
    Definir tabla, f, c Como Entero;
    Dimension tabla[3, 3];

    // Ciclos anidados para solicitar los datos al usuario
    Para f ← 1 Hasta 3 Con Paso 1 Hacer
        Para c ← 1 Hasta 3 Con Paso 1 Hacer
            Escribir "Ingrese valor para la fila ", f, " y columna ", c, ": ";
            Leer tabla[f, c];
        FinPara
    FinPara

    // Impresión en pantalla en formato de cuadrícula
    Escribir "";
    Escribir "--- MATRIZ INGRESADA ---";
    Para f ← 1 Hasta 3 Con Paso 1 Hacer
        Para c ← 1 Hasta 3 Con Paso 1 Hacer
            Escribir tabla[f, c], "  " Sin Saltar;
        FinPara
        Escribir ""; // Genera un salto de línea al finalizar cada fila
    FinPara
FinAlgoritmo
```

Si sale fuera de rango

numero2.psc

```
Algoritmo Ejercicio2_Matriz
    Definir tabla, f, c Como Entero;
    Dimension tabla[3, 3];

    // Ciclos anidados para solicitar los datos al usuario (índices 0 al 2)
    Para f ← 0 Hasta 2 Con Paso 1 Hacer
        Para c ← 0 Hasta 2 Con Paso 1 Hacer
            Escribir "Ingrese valor para la fila ", f + 1, " y columna ", c + 1, ": ";
            Leer tabla[f, c];
        FinPara
    FinPara

    // Impresión en pantalla en formato de cuadrícula
    Escribir "";
    Escribir "--- MATRIZ INGRESADA ---";
    Para f ← 0 Hasta 2 Con Paso 1 Hacer
        Para c ← 0 Hasta 2 Con Paso 1 Hacer
            Escribir tabla[f, c], " " Sin Saltar;
        FinPara
        Escribir ""; // Genera un salto de línea al finalizar cada fila
    FinPara
FinAlgoritmo
```

numero1.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    // Declaración de la matriz 3x3
    int tabla[3][3];

    // Ciclos anidados para solicitar los datos al usuario
    for (int f = 0; f < 3; f++) {
        for (int c = 0; c < 3; c++) {
            cout << "Ingrese valor para la fila " << (f + 1) << " y columna " << (c + 1) << ": ";
            cin >> tabla[f][c];
        }
    }

    // Impresión en formato de cuadrícula
    cout << endl << "--- MATRIZ INGRESADA ---" << endl;
    for (int f = 0; f < 3; f++) {
        for (int c = 0; c < 3; c++) {
            cout << tabla[f][c] << "\t"; // \t agrega una tabulación para alinear visualmente las columnas
        }
        cout << endl; // Salto de línea al terminar cada fila
    }

    return 0;
}
```

Ejercicio 3. Suma total de los elementos

Modifique el ejercicio anterior para que, después de imprimir la matriz 3x3, el programa calcule y muestre la suma total de todos los números ingresados en la matriz.

```
Algoritmo Ejercicio3_Matriz
    Definir tabla, f, c, suma_total Como Entero;
    Dimension tabla[3, 3];

    suma_total ← 0; // Inicializar el acumulador en cero

    // Ciclos anidados para solicitar los datos al usuario
    Para f ← 1 Hasta 3 Con Paso 1 Hacer
        Para c ← 1 Hasta 3 Con Paso 1 Hacer
            Escribir "Ingrese valor para la fila ", f, " y columna ", c, ": ";
            Leer tabla[f, c];
        FinPara
    FinPara

    // Impresión en pantalla en formato de cuadrícula
    Escribir "";
    Escribir "--- MATRIZ INGRESADA ---";
    Para f ← 1 Hasta 3 Con Paso 1 Hacer
        Para c ← 1 Hasta 3 Con Paso 1 Hacer
            Escribir tabla[f, c], "  " Sin Saltar;
            suma_total ← suma_total + tabla[f, c]; // Se acumula cada elemento
        FinPara
        Escribir ""; // Genera un salto de línea al finalizar cada fila
    FinPara

    // Mostrar el resultado de la suma total
    Escribir "";
    Escribir "La suma total de todos los números ingresados es: ", suma_total;
FinAlgoritmo
```

[*] numero1.cpp

```cpp
#include <iostream>

using namespace std;

int main() {
    // Declaración de la matriz 3x3 y acumulador
    int tabla[3][3];
    int suma_total = 0; // Inicializar la suma en cero

    // Ciclos anidados para solicitar los datos al usuario
    for (int f = 0; f < 3; f++) {
        for (int c = 0; c < 3; c++) {
            cout << "Ingrese valor para la fila " << (f + 1) << " y columna " << (c + 1) << ": ";
            cin >> tabla[f][c];
        }
    }

    // Impresión en cuadrícula y cálculo de la suma
    cout << endl << "--- MATRIZ INGRESADA ---" << endl;
    for (int f = 0; f < 3; f++) {
        for (int c = 0; c < 3; c++) {
            cout << tabla[f][c] << "\t";
            suma_total += tabla[f][c]; // Suma acumulativa de cada elemento
        }
        cout << endl; // Salto de línea al terminar cada fila
    }

    // Mostrar el resultado final
    cout << endl;
    cout << "La suma total de todos los números ingresados es: " << suma_total << endl;

    return 0;
}
```

Ejercicio 4. Recorrido por columnas

Por defecto recorremos las matrices fila por fila. Diseñe un programa que llene una matriz de 2 filas y 3 columnas, pero que la imprima de arriba hacia abajo (columna por columna). (Pista: invierta el orden de los ciclos Para).

PARTE B. OPERACIONES MATEMÁTICAS Y ESTADÍSTICAS

Ejercicio 5. Suma de cada fila de forma independiente

Cree una matriz de 3 filas y 4 columnas con números ingresados por el usuario. Calcule y muestre la suma de los elementos de cada fila por separado (Ej: "La suma de la fila 1 es: X").

```
Algoritmo Ejercicio5_Matriz
   Definir m, f, c, suma_fila Como Entero;
   Dimension m[3, 4];

   Para f <- 1 Hasta 3 Con Paso 1 Hacer
      Para c <- 1 Hasta 4 Con Paso 1 Hacer
         Escribir "Valor [", f, ",", c, "]: ";
         Leer m[f, c];
      FinPara
```

```
    FinPara

    Escribir "--- RESULTADOS POR FILA ---";
    Para f <- 1 Hasta 3 Con Paso 1 Hacer
        suma_fila <- 0; // Reiniciar la suma para la nueva fila
        Para c <- 1 Hasta 4 Con Paso 1 Hacer
            suma_fila <- suma_fila + m[f, c];
        FinPara
        Escribir "La suma de la fila ", f, " es: ", suma_fila;
    FinPara
FinAlgoritmo
```

Ejercicio 6. Encontrar el número mayor en la matriz

Llene una matriz de 4x4. Determine mediante un algoritmo cuál es el número más grande almacenado en toda la matriz y muestre en qué fila y columna se encuentra.

Ejercicio 7. Suma de la diagonal principal

Cree una matriz cuadrada de 3x3. Un elemento pertenece a la diagonal principal si su número de fila es exactamente igual a su número de columna (ej. [1,1], [2,2]). Sume únicamente los elementos de esta diagonal.

PARTE C. RETOS DE LÓGICA Y APLICACIÓN REAL

Ejercicio 8. Sistema de Notas por Estudiante (Matriz + Vectores)

Un profesor necesita almacenar las notas de 4 estudiantes. Cada estudiante tiene 3 notas. Utilice un vector para los nombres de los estudiantes y una matriz de 4x3 para almacenar sus notas. Calcule el promedio de cada estudiante e indíquelo en pantalla.

```
Algoritmo Ejercicio8_Notas
    Definir estudiantes Como Caracter;
    Definir notas, suma_notas, promedio Como Real;
    Definir e, n Como Entero;
    Dimension estudiantes[4];
    Dimension notas[4, 3];

    Para e <- 1 Hasta 4 Con Paso 1 Hacer
        Escribir "Nombre del estudiante ", e, ":";
        Leer estudiantes[e];
        Para n <- 1 Hasta 3 Con Paso 1 Hacer
            Escribir "Ingrese la nota ", n, " de ", estudiantes[e], ":";
            Leer notas[e, n];
```

```
    FinPara
  FinPara

  Escribir "--- INFORME DE PROMEDIOS ---";
  Para e <- 1 Hasta 4 Con Paso 1 Hacer
    suma_notas <- 0;
    Para n <- 1 Hasta 3 Con Paso 1 Hacer
      suma_notas <- suma_notas + notas[e, n];
    FinPara
    promedio <- suma_notas / 3;
    Escribir "El estudiante ", estudiantes[e], " tiene un promedio de: ",
promedio;
  FinPara
FinAlgoritmo
```

Ejercicio 9. Matriz Transpuesta

Cree una matriz A de 2 filas y 3 columnas. Luego, cree una matriz B de 3 filas y 2 columnas y transfiera los datos de A hacia B, de modo que las filas de A se conviertan en las columnas de B. Imprima la matriz resultante B.

Ejercicio 10. Cine: Venta de Asientos

Diseñe un programa que simule la sala de un cine usando una matriz de 5 filas y 5 columnas, inicializada con ceros (0 = Asiento Libre). El sistema debe permitir al usuario ingresar la fila y columna que desea comprar. Si está en 0, cambiarlo a 1 (Ocupado) y mostrar éxito. Si ya es 1, mostrar un mensaje indicando que el asiento está ocupado. El programa finaliza cuando el usuario lo indique.

4. REGLAS DE LA ACTIVIDAD

- Escribir personalmente cada algoritmo en PSeInt.
- Utilizar ciclos Para anidados correctamente estructurados.
- Terminar cada instrucción con punto y coma (;) para asegurar compatibilidad estricta.

5. PREGUNTAS DE REFLEXIÓN

1. ¿Cuál es la diferencia principal entre un vector y una matriz en PSeInt?
2. ¿Por qué siempre se requieren dos ciclos anidados para procesar completamente una matriz?
3. Si tiene una matriz dimensionada como tabla[10, 5], ¿Cuántos elementos en total puede almacenar?
4. ¿Qué ocurre si invierte el orden lógico de los ciclos (el ciclo de columnas envuelve al ciclo de filas) al procesar una matriz?

5. Describa una situación de la vida real (ej. videojuegos, sistemas empresariales) donde sería obligatorio el uso de matrices.